

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи № 2
з курсу “Операційні системи реального часу”
на тему: “Планування завдань у FreeRTOS”

Виконав:
Студент. гр. ФеП-41 Фіняк Олег
Перевірив:
Павлик М.Р.

Львів – 2025

Мета роботи. Ознайомитися з принципами планування завдань у FreeRTOS.

При завданнях з однаковим пріоритетом усі завдання виконуються рівномірно — **round-robin** (по колу).

При зворотньому порядку пріоритетів Task1 стає найчастішим у консолі, Task3 — найрідше.

якщо завдання з високим пріоритетом не викликає vTaskDelay(), то Без vTaskDelay() завдання **ніколи не передає керування**, тому FreeRTOS не може перемкнутися на інші tasks.

Завдання з найнижчим пріоритетом:

```
void vTask4(void *pvParameters) {
    const uint LED = 25;
    gpio_init(LED);
    gpio_set_dir(LED, GPIO_OUT);

    while (1) {
        gpio_put(LED, 1);
        vTaskDelay(pdMS_TO_TICKS(200));
        gpio_put(LED, 0);
        vTaskDelay(pdMS_TO_TICKS(1800));
    }
}
```

```
xTaskCreate(vTask4, "LED", 256, NULL, 0, NULL);
```

Світлодіод блимає кожні 2 секунди, і через найнижчий пріоритет **не заважає** іншим задачам.

Одне завдання з високим пріоритетом працює без vTaskDelay():

```
xTaskCreate(vTask3, "Task3", 256, NULL, 5, NULL); // Найвищий пріоритет
```

Task3 — **CPU-hog** (не віддає CPU), тому

- планувальник НЕ може переключитися
- інші задачі **НІКОЛИ** не отримують процесорний час

Дві пари задач з однаковим пріоритетом:

```
void vA1(void *p){ while(1){ printf("A1\n"); vTaskDelay(300);} }
void vA2(void *p){ while(1){ printf("A2\n"); vTaskDelay(300);} }
void vB1(void *p){ while(1){ printf("B1\n"); vTaskDelay(500);} }
void vB2(void *p){ while(1){ printf("B2\n"); vTaskDelay(500);} }
int main() {
    stdio_init_all();
    // Пара 1, пріоритет 2
    xTaskCreate(vA1, "A1", 256, NULL, 2, NULL);
    xTaskCreate(vA2, "A2", 256, NULL, 2, NULL);
    // Пара 2, пріоритет 1
    xTaskCreate(vB1, "B1", 256, NULL, 1, NULL);
    xTaskCreate(vB2, "B2", 256, NULL, 1, NULL);
}
```

```
    vTaskStartScheduler();  
}
```

У кожній групі задачі працюють **round-robin**, але група з вищим пріоритетом (A1/A2) виконується частіше.

Висновок: у лабораторній роботі було досліджено принципи планування завдань у FreeRTOS та вплив пріоритетів на їх виконання. Було показано, що завдання з вищим пріоритетом витісняють нижчі, а при однакових пріоритетах система застосовує round-robin планування. Експериментально підтверджено, що завдання без vTaskDelay() може повністю блокувати інші, не дозволяючи планувальнику виконати перемикання. Додавання нових задач та зміна пріоритетів дозволили на практиці зрозуміти механізми розподілу процесорного часу у системах реального часу.

```
// src/main.c  
#include <stdio.h>  
#include "pico/stdlib.h"  
#include "FreeRTOSConfig.h"  
#include "task.h"  
#include "pico/stdio_usb.h" // optional if using USB CDC stdin/out  
  
// Task handles (щоб мати можливість змінювати пріоритети динамічно)  
static TaskHandle_t xTask1Handle = NULL;  
static TaskHandle_t xTask2Handle = NULL;  
static TaskHandle_t xTask3Handle = NULL;  
static TaskHandle_t xTask4Handle = NULL;  
  
// Control flags  
static volatile bool high_task_busy = false; // якщо true -> Task3 не викликає  
vTaskDelay()  
static volatile uint32_t current_mode = 0;  
  
/* ----- Завдання ----- */  
  
// Низький пріоритет – виводить повідомлення кожну секунду  
void vTask1(void *pvParameters)  
{  
    (void)pvParameters;  
    for (;;)   
    {  
        printf("Task 1 (Low priority)\n");  
        vTaskDelay(pdMS_TO_TICKS(1000));  
    }  
}  
  
// Середній пріоритет – виводить повідомлення кожні 700 ms  
void vTask2(void *pvParameters)  
{  
    (void)pvParameters;  
    for (;;)   
    {  
        printf("Task 2 (Medium priority)\n");  
        vTaskDelay(pdMS_TO_TICKS(700));  
    }  
}
```

```

    {
        printf("Task 2 (Medium priority)\n");
        vTaskDelay(pdMS_TO_TICKS(700));
    }
}

// Високий пріоритет – виводить повідомлення кожні 500 ms,
// але якщо high_task_busy == true, працює без vTaskDelay() (busy loop)
void vTask3(void *pvParameters)
{
    (void)pvParameters;
    for (;;)
    {
        printf("Task 3 (High priority)%s\n", high_task_busy ? " - BUSY (no delay)"
: "");
        if (!high_task_busy)
        {
            vTaskDelay(pdMS_TO_TICKS(500));
        }
        else
        {
            // Невелика yield-операція для безпечнішого виводу,
            // але в реальному експерименті прибрати yield, щоб показати total
            starvation.
            // comment/uncomment the following line to see slightly different
            behaviors:
            // taskYIELD();
            // Якщо хочеш повністю без затримок і без yield -> закоментуй
            taskYIELD()
        }
    }
}

// LED task: блимає вбудованим LED кожні 2000 ms
void vTask4(void *pvParameters)
{
    (void)pvParameters;
    const uint LED_PIN = PICO_DEFAULT_LED_PIN;
    gpio_init(LED_PIN);
    gpio_set_dir(LED_PIN, GPIO_OUT);

    for (;;)
    {
        gpio_xor_mask(1u << LED_PIN);
        vTaskDelay(pdMS_TO_TICKS(2000));
    }
}

/* ----- Допоміжні функції для режимів експерименту ----- */

static void print_menu(void)
{
    printf("\n--- FreeRTOS scheduling lab (menu) ---\n");
}

```

```

    printf("Введіть одну з команд:\n");
    printf("  0 : Default priorities (Task3=3, Task2=2, Task1=1, LED=0)\n");
    printf("  1 : All tasks same priority (1)\n");
    printf("  2 : Reverse priorities (Task1=3, Task2=2, Task3=1, LED=0)\n");
    printf("  3 : High-priority busy (Task3 runs without vTaskDelay())\n");
    printf("  4 : Stop high-priority busy (restore delays)\n");
    printf("  m : show this menu\n");
    printf("-----\n");
}

// Встановлює пріоритети відповідно до обраного режиму
static void set_mode(uint32_t mode)
{
    current_mode = mode;
    high_task_busy = false; // дефолтна поведінка

    switch (mode)
    {
        case 0:
            // Default: Task3 high, Task2 medium, Task1 Low, LED lowest
            vTaskPrioritySet(xTask3Handle, 3);
            vTaskPrioritySet(xTask2Handle, 2);
            vTaskPrioritySet(xTask1Handle, 1);
            vTaskPrioritySet(xTask4Handle, 0);
            printf("Mode 0: Default priorities set.\n");
            break;

        case 1:
            // All same priority (1) => round-robin between Task1,2,3; LED lower so
            // runs less frequently
            vTaskPrioritySet(xTask3Handle, 1);
            vTaskPrioritySet(xTask2Handle, 1);
            vTaskPrioritySet(xTask1Handle, 1);
            vTaskPrioritySet(xTask4Handle, 0);
            printf("Mode 1: All tasks set to same priority (1).\n");
            break;

        case 2:
            // Reverse priorities: Task1 highest
            vTaskPrioritySet(xTask1Handle, 3);
            vTaskPrioritySet(xTask2Handle, 2);
            vTaskPrioritySet(xTask3Handle, 1);
            vTaskPrioritySet(xTask4Handle, 0);
            printf("Mode 2: Reverse priorities set (Task1 highest).\n");
            break;

        case 3:
            // Busy high-priority: set Task3 highest and make it busy
            vTaskPrioritySet(xTask3Handle, 3);
            vTaskPrioritySet(xTask2Handle, 2);
            vTaskPrioritySet(xTask1Handle, 1);
            vTaskPrioritySet(xTask4Handle, 0);
            high_task_busy = true;
            printf("Mode 3: Task3 set to BUSY (no vTaskDelay()). Other tasks may
            starve.\n");
            break;
    }
}

```

```

    case 4:
        // Stop busy mode – restore default priorities
        high_task_busy = false;
        vTaskPrioritySet(xTask3Handle, 3);
        vTaskPrioritySet(xTask2Handle, 2);
        vTaskPrioritySet(xTask1Handle, 1);
        vTaskPrioritySet(xTask4Handle, 0);
        printf("Mode 4: High-priority task restored to normal (with delays).\n");
        break;
    default:
        printf("Unknown mode: %u\n", (unsigned)mode);
        break;
}
}

/* ----- Менеджер меню у окремому завданні (щоб UART оброблявся в контексті
FreeRTOS) ----- */
void vMenuTask(void *pvParameters)
{
    (void)pvParameters;
    int c;

    print_menu();
    set_mode(0); // default

    for (;;)
    {
        // Читання символу з stdin з невеликим таймаутом (в мікросекундах)
        c = getchar_timeout_us(200000); // 200 ms
        if (c == PICO_ERROR_TIMEOUT)
        {
            // нічого не робимо – просто продовжуємо
            vTaskDelay(pdMS_TO_TICKS(100));
            continue;
        }
        if (c == EOF)
        {
            continue;
        }

        // Усунемо '\r' / '\n'
        if (c == '\r' || c == '\n')
        {
            continue;
        }

        switch (c)
        {
            case '0':
                set_mode(0);
                break;
            case '1':
                set_mode(1);

```

```

        break;
    case '2':
        set_mode(2);
        break;
    case '3':
        set_mode(3);
        break;
    case '4':
        set_mode(4);
        break;
    case 'm':
    case 'M':
        print_menu();
        break;
    default:
        printf("Unknown command '%c'. Натисни 'm' для меню.\n", (char)c);
        break;
    }
    vTaskDelay(pdMS_TO_TICKS(50));
}
}

/* ----- main ----- */
int main(void)
{
    stdio_init_all(); // ініціалізація UART/USB-CDC для printf і getchar

    // Дочекаємось, щоб інтерфейс stdio ініціалізувався
    sleep_ms(200);

    printf("\n=== FreeRTOS Lab: Scheduling demo ===\n");

    // Створюємо основні завдання
    // Stack size – 256 words (як у прикладі)
    BaseType_t res;

    res = xTaskCreate(vTask1, "Task1", 256, NULL, 1, &xTask1Handle);
    configASSERT(res == pdPASS);

    res = xTaskCreate(vTask2, "Task2", 256, NULL, 2, &xTask2Handle);
    configASSERT(res == pdPASS);

    res = xTaskCreate(vTask3, "Task3", 256, NULL, 3, &xTask3Handle);
    configASSERT(res == pdPASS);

    res = xTaskCreate(vTask4, "LED", 256, NULL, 0, &xTask4Handle);
    configASSERT(res == pdPASS);

    // Створимо окреме завдання для меню/введення
    TaskHandle_t menuHandle = NULL;
    res = xTaskCreate(vMenuTask, "Menu", 512, NULL, 1, &menuHandle);
    configASSERT(res == pdPASS);

```

```
// Запускаємо планувальник
vTaskStartScheduler();

// До сюди виконання не доходить, якщо планувальник запущено правильно
for (;;)
{
    // Якщо щось пішло не так — застрягаємо тут
}

return 0;
}
```